Experimental Practice - Stable Matching

Beatriz Mendoza Hernández
Design and Analysis of Algorithms , Center for Computing Research
Instituto Politécnico Nacional, CDMX
bmendozaa26@cic.mx

***Abstract*—The Stable Matching Problem is a classical problem in algorithm design that aims to produce stable pairings between two equally sized sets based on preference rankings. This paper presents the implementation and empirical temporal analysis of the Gale–Shapley algorithm using varying input sizes. The experiments were automated to measure the reading time, algorithm execution time, and output generation time independently. The results confirm the theoretical quadratic time complexity $O(n^2)$ and demonstrate that the selection of the proposer affects the optimality of the matching but not the asymptotic performance.**

***Index Terms*—Stable Matching, Gale-Shapley Algorithm, Experimental Algorithm Analysis, Time Complexity.**

I. Introduction

Matching problems appear in multiple real-world applications, such as school admissions, job assignments, and resource allocation systems. The Stable Matching Problem seeks to match elements from two sets while avoiding instability, defined as the existence of a pair that prefers each other over their assigned partners.

The Gale–Shapley algorithm provides a deterministic solution guaranteeing stability. This work implements the algorithm in Python and evaluates its temporal behaviour experimentally. The study analyses execution performance under increasing input sizes and compares results when either men or women initiate proposals.

A. Objectives

This practice aims to experimentally verify the asymptotic behavior of the Gale–Shapley algorithm, which computes a stable matching between two sets of men and women with equal cardinality N. In other words, the goal is to solve the Stable Matching Problem (SMP) in its basic formulation.

B. Specific Objectives

1) Implement the Gale–Shapley algorithm using a programming language.
2) Generate M input files to be used as datasets for the algorithm.
3) Execute the Gale–Shapley algorithm for each input file and measure its execution time.
4) Store the measured execution times for subsequent analysis.

C. Description of Activities

1) Methodology: The experimental methodology consisted of implementing and evaluating the Gale–Shapley stable matching algorithm under different input sizes in order to analyze its temporal behavior.

First, two preference sets were defined representing two groups of participants (men and women), each containing N elements. Every participant provided a complete ordered list of preferences over the members of the opposite group.

The Gale–Shapley algorithm was implemented following the proposal-based mechanism. At each iteration, a free proposer selects the next candidate according to its preference list. If the receiver is free, a tentative engagement is formed. Otherwise, the receiver compares the new proposal with its current partner using a precomputed ranking structure and keeps the preferred option. This process continues until no free proposers remain, guaranteeing a stable matching.

To analyze performance, multiple input files with increasing values of N were generated. Each file was executed independently, and execution times were measured and stored for later analysis.

The execution time was divided into three stages:

- Reading time: loading and parsing input data.
- Algorithm time: execution of the Gale–Shapley procedure.
- Printing time: generation of the output matching.

Timing measurements were obtained using high–precision timers, allowing an empirical comparison with the theoretical time complexity $O(n^2)$.

2) Tools: The implementation was developed using the Python programming language. The following data structures and tools were used:

Data Structures:

- Dictionaries (`dict`) to store preference lists and matchings.
- Lists (`list`) to manage free participants and ordered preferences.
- Strings to represent participant identifiers.

Libraries and Functions:

- `time.perf_counter()` for high-resolution execution time measurement.
- `subprocess.run()` to execute the algorithm automatically for multiple input files.
- `csv` module to store experimental results in CSV format.
- `re` (regular expressions) to extract timing values from program output.
- `matplotlib.pyplot` to generate linear and log-log performance graphs.

- `pandas` to load and process experimental data.

Input Generation:

- Input files were generated programmatically as text files containing preference lists.
- Each file represents a different problem size N and was stored inside the `inputs/` directory.

Time Measurement Process:

- The timer started before reading the input data.
- Independent timers measured reading, algorithm execution, and output printing.
- Results were exported automatically into a CSV file used later for visualization and analysis.

3) Gale–Shapley Algorithm: The stable matching is computed using the classical Gale–Shapley proposal algorithm. The pseudocode used in this work is presented below.

[H] Gale–Shapley Stable Matching

```
Input: Preference lists for men and women
Initialize S to empty matching
while some man m is unmatched and has not proposed to every woman do
    w ← first woman on m's list to whom m has not yet proposed
    if w is unmatched then
        Add pair m–w to matching S
    else if w prefers m to her current partner m' then
        Remove pair m'–w from matching S
        Add pair m–w to matching S
    else
        w rejects m
    end if
end while
return stable matching S
```

=0

II. Results

The experimental evaluation of the Gale–Shapley algorithm was conducted using automatically generated input instances of increasing size N. For each input file, the following execution times were measured:

- Data reading time
- Matching computation time
- Output printing time
- Total execution time

The experiments were executed sequentially using an automated script that processed all input files stored in the `inputs` directory. The results were stored in a CSV file and later used to generate graphical representations.

Fig. 1 shows the execution times measured on a linear scale. The algorithm execution time dominates the total runtime as the input size increases.

To analyze asymptotic behavior, a logarithmic scale was applied to both axes. Fig. 2 presents the log–log representation of total execution time.

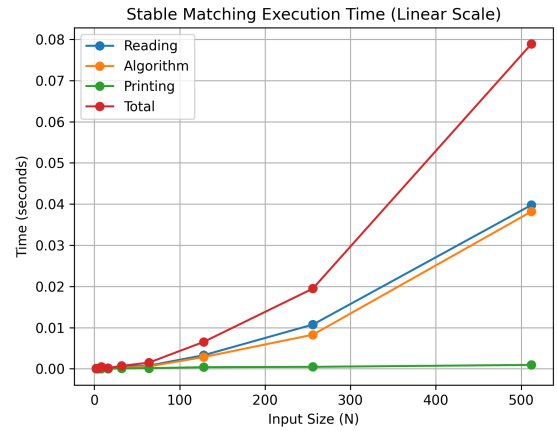

Fig. 1. Execution time of the Gale–Shapley algorithm for increasing input sizes (linear scale).

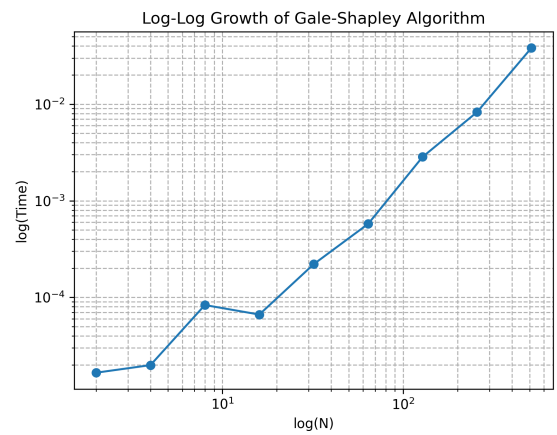

Fig. 2. Log–log plot of execution time showing quadratic growth behavior.

III. Analysis of Results

The experimental results confirm the theoretical time complexity of the Gale–Shapley algorithm. According to the algorithmic analysis, each proposer may propose to every receiver at most once, producing a worst-case complexity of $O(N^2)$.

In the linear-scale graph, execution time increases rapidly as the input size grows. The dominant contribution corresponds to the matching computation phase, while reading and printing times remain comparatively small.

The log–log graph provides stronger evidence of asymptotic behavior. The approximately linear trend observed in Fig. 2 indicates a polynomial relationship between execution time and input size. Additionally, experiments were conducted considering both proposing scenarios (men proposing and women proposing). The results show that proposer selection affects the resulting stable matching but does not modify the asymptotic runtime complexity of the algorithm.

Therefore, empirical measurements validate the theoretical

expectations of the Stable Matching Problem solution.

IV. Conclusions

This work presented the implementation and experimental evaluation of the Gale–Shapley stable matching algorithm. An automated experimental script was developed to generate input instances, execute the algorithm, measure execution times, and visualize performance growth.

The obtained results experimentally verify the quadratic time complexity $O(N^2)$ predicted by theoretical analysis. The log–log graphical representation demonstrated a clear polynomial growth consistent with the expected asymptotic behavior.

Although the choice of proposer (men or women) changes the optimality of the resulting matching for each group, it does not affect the computational complexity of the algorithm.

V. References

D. Gale and L. S. Shapley, "College Admissions and the Stability of Marriage," American Mathematical Monthly, vol. 69, no. 1, pp. 9–15, 1962.